EduCore — How the Project Fits Together

Part 1 of 5: orientation, layers, and the life of a request

EDUCORE BACKEND · .NET 10 · ASP.NET CORE · EF CORE 10 · POSTGRESQL

1. What the application is

EduCore is a REST API for running schools. It is **multi-tenant**: one deployment, one database, many schools, and no school can ever see another's data. That single requirement shapes almost every design decision in the codebase, so it is the thing to understand first.

The domain it models:

CONCEPT	MEANING
`School`	The tenant. Everything else belongs to exactly one.
`AcademicYear` / `GradingPeriod`	The calendar. A year (2025/2026) contains terms.
`Subject`	Maths, History. Defined per school.
`Class`	A group of students in one year, e.g. “10A”.
`Student`, `Teacher`, `Admin`	People. Each owns a `User` row that holds their login.
`Parent`	A guardian contact attached to a student. Not a login.
`Enrollment`	A student's placement in a class, with a start and optional end date.
`TeacherSubject`	A qualification: this teacher may teach this subject.
`ClassSubjectTeacher`	An assignment: this teacher teaches this subject *to this class*.
`DiaryEntry`	One lesson that actually happened. The “digital diary”.
`Attendance`	One student's presence at one lesson, plus behaviour notes.
`Mark`	A grade, tied to an enrollment, a subject and a grading period.
`AuditLog`	A record of every change, written automatically.

The four roles

ROLE	BELONGS TO A SCHOOL?	CAN DO
`SuperAdmin`	No	Create schools, appoint school admins, read the audit log.
`Admin`	Yes, one	Everything inside their own school.
`Teacher`	Yes, one	Log lessons, mark attendance, give marks — for their own classes.
`Student`	Yes, one	Read their own record and their own results.

Note the asymmetry: `SuperAdmin` has *no* school. That is deliberate and it explains several pieces of code later on. The SuperAdmin is an operator of the platform, not a member of any school.

2. The folder map

Every folder has one job. If you can say which folder a piece of code belongs in, you understand the architecture.

FOLDER	HOLDS	RULE
`Models/`	Entity classes — one per database table.	Plain properties, no logic.
`Data/`	`AppDbContext` and the two interceptors.	How the app talks to Postgres.
`DTOs/`	The shapes that cross the network.	Records. Never entities.
`Repositories/`	Database queries.	No business rules, no authorization.
`Services/`	Business rules and authorization.	The brain. No HTTP, no SQL.
`Endpoints/`	URL → service call.	Thin. No logic at all.
`Security/`	Who is calling, and the role policies.	Reads the token, nothing more.
`Validation/`	FluentValidation rules per DTO.	Shape only, never database lookups.
`Mapping/`	The AutoMapper profile.	Entity ↔ DTO conversions.
`Exceptions/`	Custom exceptions + the global handler.	Turns thrown exceptions into HTTP status codes.
`Migrations/`	Generated schema history.	Never hand-edited.

> **The one rule that keeps it straight.** Dependencies point in one direction: `Endpoints → Services → Repositories → AppDbContext`. A repository must never call a service. An endpoint must never touch `AppDbContext`. If you ever need to break that, something is in the wrong folder.

3. The three layers of a request

Every layer answers a different question, and they compose. This is the single most important diagram in the project:

```
HTTP request ↓ 1. AUTHENTICATION — is this token real? → 401 if not ↓ 2. AUTHORIZATION (role) — may this role open → 403 if not this endpoint at all? ↓ 3. VALIDATION — is the request body well-formed? → 400 if not ↓ 4. SERVICE — business rules • AccessGuard: is this specific row yours? → 403 if not • domain rules: does this make sense? → 400/409 if not ↓ 5. REPOSITORY — the SQL query • query filter silently adds: AND SchoolId = yours ↓ 6. SAVE — interceptors run • SchoolScopeInterceptor: refuses cross-school writes • AuditSaveChangesInterceptor: adds AuditLog rows ↓ HTTP response
```

Layers 2, 4 and 5 are three *different* security questions and it is worth being precise about them, because they are easy to confuse:

- **Role** (“teachers may read marks”) cannot say *which* marks.

- **School filter** (“only your school”) cannot stop one teacher reading another teacher's class, because they are in the same school.
- **AccessGuard** answers exactly that last question, and nothing else.

4. The life of one request, concretely

Take `GET /api/classes/{classId}/marks` with a teacher's token.

Step 1 — the endpoint

`Endpoints/ClassEndpoints.cs`

```
group.MapGet("/{classId:guid}/marks",
        async (Guid classId, Guid? subjectId, Guid? gradingPeriodId,
               IMarkService service, CancellationToken ct) =>
            Results.Ok(await service.GetByClassAsync(classId, subjectId, gradingPeriodId, ct)))
    .RequireAuthorization(Policies.AdminOrTeacher);
```

Three things are happening. `{classId:guid}` means the URL segment must parse as a GUID or the route does not match at all. `IMarkService service` is **dependency injection** — you never write `new MarkService(...)`; ASP.NET Core builds it because `Program.cs` registered it. And `.RequireAuthorization` is layer 2: a student's token gets a 403 here, before a single line of your code runs.

Step 2 — the service

`Services/MarkService.cs`

```
public async Task<IEnumerable<MarkListDto>> GetByClassAsync(
    Guid classId, Guid? subjectId, Guid? gradingPeriodId, CancellationToken ct)
{
    await access.EnsureCanViewClassAsync(classId, ct);
    return await marks.GetByClassAsync(classId, subjectId, gradingPeriodId, ct);
}
```

One line of authorization, one line of work. `EnsureCanViewClassAsync` throws `ForbiddenException` if this teacher has nothing to do with this class. Nothing catches it here — see step 5.

Step 3 — the repository

The repository builds the SQL. You wrote `Where(m => m.ClassId == classId)`, but the SQL that reaches Postgres also contains `AND m."SchoolId" = @schoolId`, which you did not write. EF Core adds it because of the global query filter in `AppDbContext`. That is covered in Part 2.

Step 4 — the response

The repository returns `MarkListDto` objects, not `Mark` entities. `Results.Ok(...)` serialises them to JSON.

Step 5 — what if something threw?

`Exceptions/GlobalExceptionHandler.cs`

```
var (statusCode, error, message) = exception switch
{
    NotFoundException          => (404, "Not Found",         exception.Message),
    UnauthorizedException      => (401, "Unauthorized",      exception.Message),
    RequestValidationException => (400, "Validation Failed", exception.Message),
    BusinessException          => (400, "Business Error",    exception.Message),
    ConflictException          => (409, "Conflict",          exception.Message),
    ForbiddenException         => (403, "Forbidden",         exception.Message),
    _                          => (500, "Internal Server Error", "An unexpected error occurred.")
};
```

This is why services just `throw` and never build HTTP responses. One handler maps every exception type to a status code and a consistent JSON body. Note the last line: an *unexpected* exception logs the real detail server-side but tells the caller nothing — a stack trace in an API response is an information leak.

5. Why DTOs exist

A DTO (Data Transfer Object) is the shape that crosses the network. Entities never do. Three reasons, and you should be able to give all three:

1. **Security.** `User` has a `PasswordHash`. Serialise the entity and you have published it.
2. **Control.** `StudentListDto` deliberately omits `PersonalNumber` (a national ID) so a list of 500 students is not a bulk export of national IDs. `StudentDetailDto` includes it, because looking at one student is a different act from listing everyone.
3. **Decoupling.** Renaming a database column should not break the front end.

The naming convention is consistent throughout: `XCreateDto` comes in to create, `XEditDto` comes in to update, `XListDto` goes out in lists, `XDetailDto` goes out for one record.

6. The packages, and what each is actually for

PACKAGE	JOB
`Microsoft.EntityFrameworkCore`	The ORM. Turns LINQ into SQL and rows into objects.
`Npgsql.EntityFrameworkCore.PostgreSQL`	The PostgreSQL driver for EF Core.
`Microsoft.AspNetCore.Authentication.JwtBearer`	Validates the `Authorization: Bearer` token on every request.
`AutoMapper`	Converts entities to DTOs and DTOs to entities.
`FluentValidation`	Declares validation rules as code instead of attributes.
`PasswordGenerator`	Generates the temporary password on an admin reset.
`Microsoft.AspNetCore.OpenApi`	Generates the API description used by Swagger-style tooling.

Password *hashing* is not in that list. It comes from `PasswordHasher<User>`, which ships inside ASP.NET Core — see Part 3.

7. Reading order when you are lost

Any feature can be traced in the same four steps. To answer “how does giving a mark work?”:

1. Find the URL in `Endpoints/`. It tells you the service method and the required role.
2. Read that method in `Services/`. This is where all the rules live.
3. Follow any repository call into `Repositories/` if you need the query.
4. Check `DTOs/` for the exact request and response shape.

Part 4 of this series does exactly that for four real requests, line by line.